RRC Program Architecture Analysis

1. Project Architecture Diagram

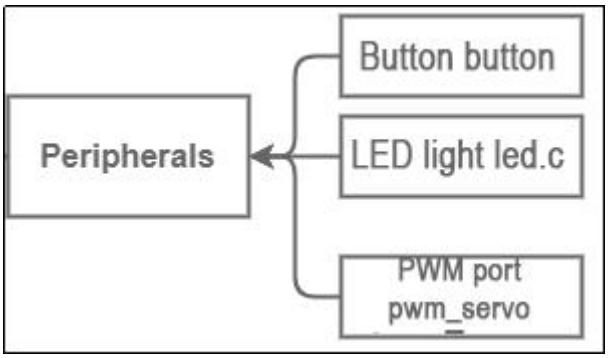
The project structure analysis diagram is a functional architecture diagram of the robot car. It outlines the main source code files that the robot car will use. As the original image is too long, it would take up a lot of space. You can refer to the "RRC Project Structure Diagram.png" file in the current folder for comparison. The contents of the files involved are located in the "Attachments/Factory Examples" folder.

2. Architecture Analysis

The RRC program mainly consists of four main parts: Peripherals, Chassis, Porting, and System.

Next, each part will be explained.

2.1 Peripherals



Peripherals involves the software initialization and execution functions of the relevant components on the STM32 main control board, including:

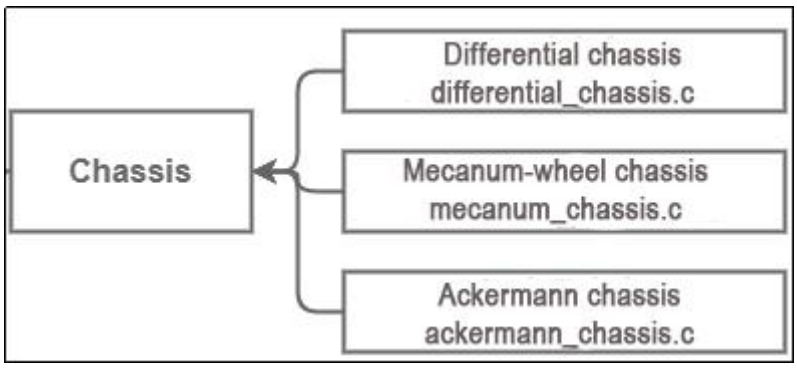
Button (button.c): This file is used to implement hardware-independent button

detection. The callback function for button presses is defined here. It enables implementing button detection functionality across different hardware platforms using the same code, requiring only a specific implementation for reading button states. It also supports various button events.

LED (led.c): This file contains hardware- and system-independent code for LED functionality. It defines parameters, functions, and tasks related to LED operation.

PWM Interface (pwm_servo.c): This file contains hardware-independent code for PWM servo control. It provides an interface to control PWM servos, allowing them to move to specific positions within specified time frames and enabling deviation correction.

2.2 Chassis



The Chassis folder contains implementations of kinematic algorithms for different types of chassis (velocity, acceleration, turning radius), including:

Differential Chassis (differential_chassis.c): Implementation of kinematic code for a differential drive chassis. The main functions to focus on are diff_chassis_move and set_velocity_radius.

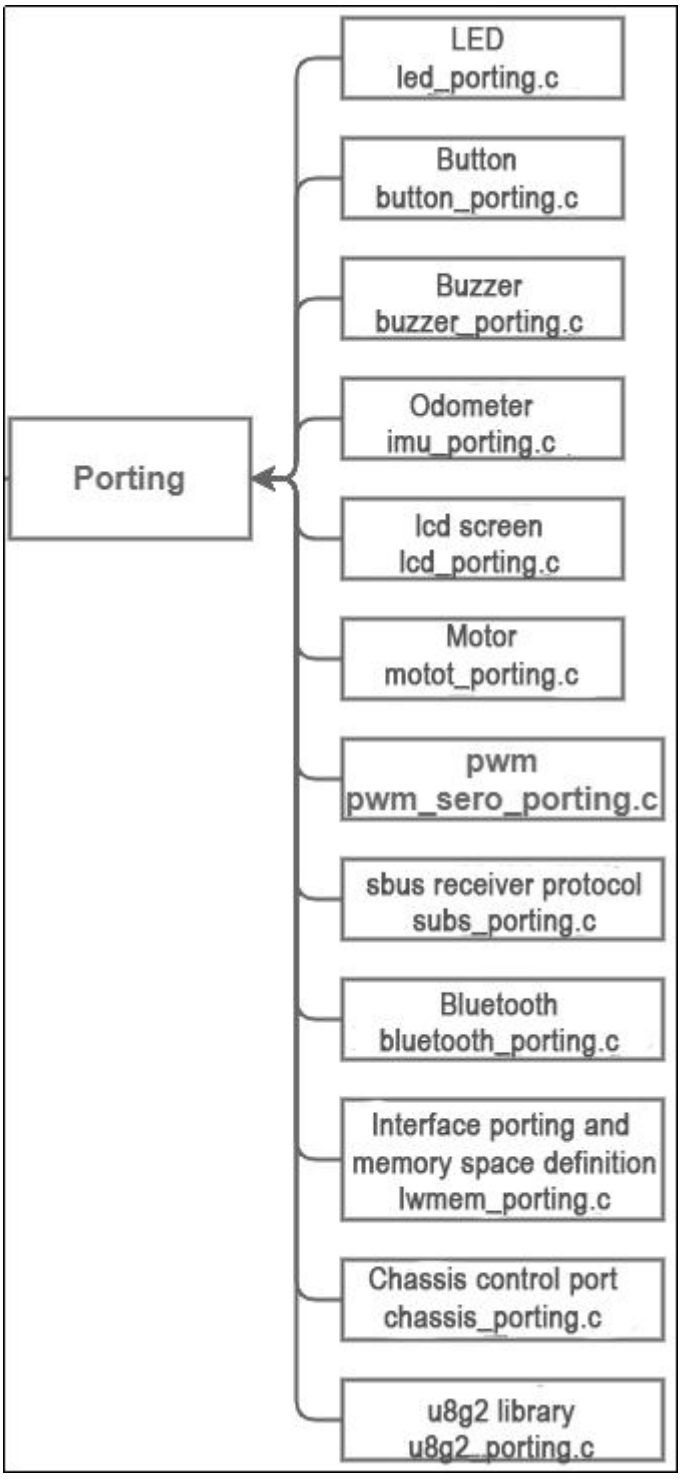
Mecanum Chassis (mecanum_chassis.c): Implementation of kinematic code for a mecanum drive chassis. The main functions to focus on are

mecanum_chassis_set_velocity, mecanum_chassis_set_xy, and set_velocity_radius.

Ackermann Chassis (ackermann_chassis.c): Implementation of kinematic code for an Ackermann drive chassis. The main functions to focus on are ackermann_velocity_difference, ackermann_chassis_move, and set_velocity_radius.

2.3 Porting

Porting involves the implementation functions for the interfaces of components on the STM32 main control board and related sensor accessories that can be mounted. When calling these functions later on, particular attention can be paid to the "hiwonder/Porting" folder.



LED (led_porting.c): Example and interface for controlling onboard LED lights.

The main functions to focus on are:

- leds_init: Used to initialize LED-related memory and variables.
- led_set_pin: Used to set the state of the LED lights (on/off).

Button (button_porting.c): Example and interface for controlling onboard buttons. The main functions to focus on are:

- buttons_init: Used to initialize button-related memory and variables.
- button_timer_callback: This is a timer callback function used for periodically scanning the state of the buttons.

Buzzer (buzzer_porting.c): Example and interface for controlling the onboard buzzer. The main functions to focus on are:

buzzers_init: Used to initialize buzzer-related memory and variables.

buzzer1_set_pwm: Buzzer PWM setting interface used to set the PWM frequency of the buzzer.

Odometer (imu_porting.c): Interface file for MPU6050, retrieving the three-axis angular values of the IMU odometer. The main functions to focus on are:

imus_init: Used to initialize the IIC address of MPU6050 and related parameters.

imu_task_entry: IMU task entry function. In the loop, it waits for a semaphore, then updates the IMU data, retrieves acceleration and gyroscope data, and sends these data packets.

LCD Screen (lcd_porting.c): Interface file for the LCD screen. The main functions to focus on are:

lcds_init: Initialization function for the LCD screen. Used to initialize parameters such as screen dimensions, write frequency, and sleep frequency.

lcd1_spi_write: Writing content to the LCD screen.

Motor Driver (motor_porting.c): Used to drive the motors for rotation.

The main functions to focus on are:

motors_init: Initialization function for the motors. Used to initialize parameters such as PID parameters and rotation speed limits.

set_motor_param: Writing PID parameters for the motors.

PWM (pwm_servo_porting.c): Used for controlling PWM servos.

The main functions to focus on are:

pwm_servos_init: Initialization function for the PWM interface. This function initializes four PWM servo objects and configures a timer to generate PWM signals. Each servo object has a corresponding write function associated with it. These functions send PWM signals to the physical pins of the servo, thereby controlling its rotation.

pwm_servo1_write_pin: Control function for PWM servo 1.

SBUS Receiver Protocol (sbus_porting.c): Used for receiving the SBUS protocol.

The main functions to focus on are:

sbus_init: Initializes SBUS-related memory and variables and starts SBUS reception.

sbus_dma_receive_event_callback: SBUS reception event callback function. Refer to the code comments for specific details.

Bluetooth (bluetooth_porting.c): Used for implementing the Bluetooth serial protocol transmission and reception interface.

The main functions to focus on are:

- bluetooth_task_entry: Bluetooth task entry function, executed using FreeRTOS. In the loop, it waits for the Bluetooth to send an idle

semaphore, retrieves data from the Bluetooth data transmission queue, and sends data using DMA.

- bluetooth_dma_rx_event_callback(UART_HandleTypeDef *huart, uint16_t length): Bluetooth reception event callback function.
- bluetooth_dma_tx_complete_callback(UART_HandleTypeDef *huart): Callback function for Bluetooth DMA transmission completion.

Interface porting and memory space definition (lwmem_porting.c): Definition of memory space addresses.

Array of lwmem_region_t structures: Each structure defines a memory region. The first region starts at address 0x10000000 with a size of 64KB, used for RAM on the microcontroller. The second region is lwmem_ram, defined earlier, with a size of 38KB. The last item { NULL, 0 } serves as the end marker for the array.

Chassis Control Interface (chassis_porting.c): Used for controlling chassis movement. It defines objects for five different types of mobile chassis, provided for user selection. When using it, users can choose the appropriate chassis for control.

```
DifferentialChassisTypeDef jetank;
DifferentialChassisTypeDef tank_black;
DifferentialChassisTypeDef ti4wd;
MecanumChassisTypeDef      jetauto;
AckermannChassisTypeDef   jetacker;
```

- chassis_init: Initializes the chassis.
- set_chassis_type: Selects the type of chassis.

---

u8g2 (u8g2_porting.c): Used for driving OLED screens with the u8g2 library. It serves as the OLED screen driver interface.

u8g2_init: Initializes the u8g2 object, registers relevant driver interfaces, and

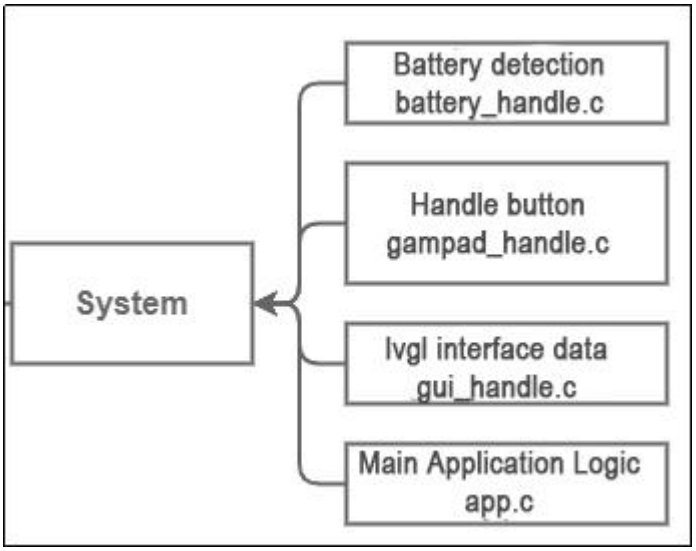
initializes the screen.

u8x8_gpio_and_delay: This function provides GPIO and delay operations for the u8g2 library.

u8x8_byte_hw_i2c: This function is a hardware-level interface of the u8g2 library used for implementing I2C communication with the OLED screen.

2.4 System

The "System" folder contains system files responsible for initiating various tasks and other component-related initializations. It is mainly divided into four parts: battery detection, gamepad buttons, lvgl interface data, and main application logic.



Battery Detection (battery_handle.c): Used for detecting battery voltage, calculating actual voltage, and sending low voltage alerts.

- battery_check_timer_callback function: Key aspects include how to handle and report battery voltage, as well as how to respond to low voltage situations. This function covers several key functionalities: 1. Reading and filtering battery voltage. 2. Periodically sending battery voltage to GUI or Bluetooth. 3. Low battery alerts.

Gamepad Buttons (gamepad_handle.c): Used for receiving button messages from PS2 gamepads and sending message handles.

USBH_HID_EventCallback function: Callback function for handling gamepad detection events. It connects and controls the gamepad.

A_T_C(int8_t analog_x, int8_t analog_y) function: This function interprets the position of the analog joystick based on its x and y values. It handles and interprets USB HID events, translating analog joystick movements from the gamepad into specific character directions.

LVGL (gui_handle.c): Used for updating data, information, and display content in the LVGL interface.

update_sbus_view function: Displays SBUS protocol messages.

lvgl_timer_callback function: Callback function for LVGL events.

gui_task_entry function (compiled only when ENABLE_LVGL is true): The main loop waits for and processes events in the message queue. Depending on the event type, it can update battery voltage display or handle other LVGL tasks.

Main Application Logic (app.c): The startup file for the main application program.

This typically contains major task entry functions, tasks created in FreeRTOS, and message queue tasks. This program describes the main application logic of a robot, involving hardware initialization, configuration, and response to button events.

Note: Before making any modifications to the program, be sure to back up the original factory program. Avoid making direct modifications to the source code files to prevent any accidental parameter changes that may lead to robot malfunctions

and become irreparable!!!